Boosting Academic Productivity with AI

Claude Code Workshop

Dr Alberto Ardid
AI (fake) guru

--
Dr Alberto Ardid
Lecturer / Assistant Professor
Department of Civil and Environmental Engineering
University of Canterbury, New Zealand

E319 |alberto.ardid@canterbury.ac.nz

First look: launching Claude Code

LIVE NOW

In the terminal:

```
$ claude
> hello, what can you see in this folder?
```

What happens next:

→ Claude explores your directory tree

→ It reads key files to understand the project

→ It answers with context - not generic advice

This is what 'context-aware' looks like.

> I'm running a data analysis workshop for civil engineering academics. I need one infrastructure example that everyone in the room will instantly understand and that has publicly available inspection data. What should I use?

Prompt 1 - Download Data

Go to the FHWA National Bridge Inventory ASCII data page and download the most recent delimited data file for Texas. Create the directory bridge_risk_demo and save the dataset as bridge_risk_demo/bridges_texas.csv.

Steps to follow:

1 *Find the FHWA NBI ASCII data download page (search for "FHWA National Bridge Inventory ASCII" if needed)*

2 *Locate the Texas state file for the most recent available year*

3*Download the file - it may be a .zip archive containing a .txt or .csv file*

4 *Extract it and save the data as bridge_risk_demo/bridges_texas.csv*

5 *If the file is pipe-delimited (|), re-save it as a comma-separated CSV*

Once the file is saved, report:

- *Total number of bridge records*
- *Number of columns*
- *Year of the data*
- *The 10 most relevant column names for a structural condition and traffic analysis*
- *Any immediate data quality concerns (e.g. columns that are mostly empty)*

Claude.ai browser vs Claude Code CLI

Feature	Claude.ai (browser) ~ ChatGPT chat	Claude Code (CLI)
Interface	Chat window	Terminal / command line
File access	Upload only	Full local filesystem
Code execution	None	Yes (with your approval)
Automation	Manual only	Scriptable, reusable
Context	Single conversation	Reads your whole project
Best for	Quick Q&A, drafting	Complex workflows, projects

Basic CC tools: Interact with files. Read files. Write/update files. Upload/download content. Write code. Execute code. Analyse outputs.

Browser AI helps you *think faster*.
Claude Code helps you *do work faster*.

Why Claude Code feels different

It works inside your environment.

- ✓ Reads your existing files - lectures, data, scripts, papers
- ✓ Remembers the context of your whole project, not just one message
- ✓ Writes code that fits your folder structure and naming conventions
- ✓ Iterates: you say 'now shorten it'; it knows what 'it' is

That continuity is what makes it feel less like a tool… and more like a **collaborator**.

Prompt 2 - Explore dataset (structured data analysis)

Read bridge_risk_demo/bridges_texas.csv. Create the directory 'outputs' inside the folder 'bridge_risk_demo', and produce a structured data description saved to bridge_risk_demo/outputs/data_description.md. The report should contain:

1. Dataset overview- Source: FHWA National Bridge Inventory- State: Texas (state code 48)- Number of records- Number of columns- Year of data- File size

2. Key columns for our analysis. For each of the following NBI fields, provide: column name as it appears in the file,NBI item number, plain-English description, data type, valid range or unique values,and percentage of missing/null values. Fields to describe:- Structure identifier (STRUCTURE_NUMBER_008)- County (COUNTY_CODE_003)- Coordinates (LAT_016, LONG_017)- Year built (YEAR_BUILT_027)- Average daily traffic (ADT_029)- Deck condition rating (DECK_COND_058)- Superstructure condition rating (SUPERSTRUCTURE_COND_059)- Substructure condition rating (SUBSTRUCTURE_COND_060)- Culvert condition (CULVERT_COND_062)- Main span material (STRUCTURE_KIND_043A)- Scour criticality (SCOUR_CRITICAL_113) . Determine the encoding format of LAT_016 and LONG_017 (e.g., decimal degrees, decimal degrees × 10^6, or DDMMSSCC packed format). Show a worked example converting one raw value to decimal degrees.

3. Data quality summary- Which key columns have missing values, and how many?- Are there out-of-range condition ratings (valid range: 0-9)?- Are there implausible year_built values (before 1800 or after current year)?- Are there negative or zero ADT values that should be excluded?- Recommended cleaning steps before analysis

4. Five observations for a bridge engineer List five specific, quantitative observations about this dataset that would beinteresting to a bridge structural or asset management engineer. Include numbers. Examples: oldest bridge still open, county with most poor-condition bridges,percentage of bridges exceeding design life, etc.

Plan first. Execute second.

Claude Code has a plan mode - it maps out the full job before writing a single line.

1. Plan (no execute)

claude --plan
Claude maps out every step, file, and decision.
Nothing is written or run yet.

2. Review the plan

Read it. Does the logic hold?
Ask: what could go wrong here?
Change scope before any code exists.

3. Update the plan

Paste feedback back in.
Claude revises the approach.
Repeat until you're confident.

4. Execute

claude --resume (or paste plan back).
Now it builds - with a validated blueprint.

The habit: never execute a big job without reviewing the plan first.
Much cheaper to fix thinking than to fix code.

Don't use AI to *generate answers*.
Use it to *design solutions first*.

Prompt 3 - Plan Mode

Before writing any code, read bridge_risk_demo/outputs/data_description.md and produce a detailed implementation plan for bridge_analysis.py. Structure the plan using the four steps below. For each step, list: - the specific functions you will write (name + one-line purpose) - the key libraries/methods you will use - any assumptions or edge cases you need to handle - any potential pitfalls to watch for
STEP 1 - DATA LOADING & CLEANING ---Plan how to load bridge_risk_demo/bridges_texas.csv, select the requiredNBI columns, and apply all cleaning filters (missing values, conditionrating bounds [0,9], YEAR_BUILT in [1800,2024], ADT >= 0).
*STEP 2 - RISK SCORE COMPUTATION ---Plan how to compute condition_risk, age_risk, and traffic_risk, andcombine them into risk_score using the weighted formula: risk_score = 0.20*condition_risk + 0.6*age_risk + 0.2*traffic_risk. Note where log1p normalisation is applied and how max_ADT is derived. Plan how to classify bridges into Low / Moderate / High / Critical tiers.*
STEP 3 - TERMINAL SUMMARY ---Plan the printed output: total bridges, mean/median risk, tier countsand percentages, poor-condition count (min_cond <= 4), oldest bridge, and the top-5 highest-risk bridges table.
STEP 4 - FIGURES ---For each of the four figures (fig1_risk_map, fig2_age_condition,fig3_risk_distribution, fig4_county_risk), describe: - the matplotlib calls needed - the colormap and axis configuration - any non-obvious formatting (FHWA tick labels, county filtering, etc.)
CODE QUALITY CHECKLIST ---Confirm your plan addresses:
Module-level docstring with research question, NBI references, formula Docstring on every function
All file paths as pathlib. Path constants at the top
Comments on every non-obvious calculation step
matplotlib only (no seaborn, no plotly)
All figures saved at dpi=150
Script runnable with: python bridge_analysis.py
Do NOT write any code yet. Output the plan only. save the plan in bridge_risk_demo/outputs/plan.md

Texas Bridge Risk Assessment

Research question: Which Texas bridges carry the highest combined risk?

Real open data	FHWA National Bridge Inventory - Texas bridges, 120+ fields per bridge
Composite risk score	Condition (20%) + Age vs design life (60%) + Traffic exposure (20%)
Condition ratings	NBI 0-9 scale: 9 = excellent, 4 = poor, 2 = critical - some TX bridges are at 2 right now
Four output figures	Risk map (4-panel: full TX + DFW/Houston/Austin zooms, tiered colorbar, critical bridges highlighted) · Age vs condition · Risk distribution · County breakdown
Plain English output	Claude translates terminal statistics into abstract-ready bullet points

Extra useful step: Explain

Download

One prompt.
Claude finds the file on a government website, downloads and unpacks it.

Plan

Plan mode - map out the full analysis before writing a single line of code.

Run

Execute the script.
Generate figures.

Explain

Ask Claude to translate the terminal output into scientific language.

Prompt 3 - Generate python codes and run them (by accepting the plan)

Run the plan

The Full Pattern for Plan Mode

Step 1	**claude --plan**	Ask Claude Code to map out the big job in plan mode. No execution.
Step 2	**Paste plan → Claude.ai Deep Research**	Ask for: critique · validation · implementation examples · pros & cons · known failures.
Step 3	**Paste research report → Claude Code**	Ask it to update and improve the plan based on what the research found.
Step 4	**Execute the updated plan**	Now build - with a battle-tested blueprint.

Today: abbreviated version - plan → review together → execute.

Extra Prompt - Review and Update Plan

Here is the Claude Code plan to review and improve: [PASTE PLAN HERE]
Review the following Claude Code plan critically and improve it using deep research.

What I want from you:
** Evaluate whether the plan is technically sound, complete, and realistic.*
** Identify weak assumptions, missing steps, risks, dependencies, and unclear reasoning.*
** Check whether the proposed approach is the best one, and suggest better alternatives where appropriate.*
** Improve the plan so it is more robust, efficient, and executable.*
** Use deep research to validate recommendations, best practices, relevant tools, architecture choices, and implementation strategy.*
** When useful, compare options and explain tradeoffs.*

Output format:
*1. **Executive summary** - what is good, what is weak, and the biggest improvements needed.*
*2. **Critical review of the original plan** - issues, gaps, risks, and questionable assumptions.*
*3. **Improved plan** - a rewritten and better version of the plan.*
*4. **Recommended implementation details** - tools, methods, architecture, workflow, and safeguards.*
*5. **Open questions / decisions needed** - anything that still requires clarification.*
*6. **Research-backed rationale** - explain why your recommendations are stronger.*

Instructions:
** Be rigorous, skeptical, and practical.*
** Do not just lightly edit the plan; substantially improve it where needed.*
** Preserve the original objective unless the objective itself is flawed.*
** Flag anything that seems overengineered, underspecified, risky, or unlikely to work.*
** Make the final version actionable and structured.*

How to write better prompts (the shortcut I use)

Don’t optimise prompts in your head. Use AI to optimise AI.

My workflow:

1. Write a rough prompt (messy is fine)
2. Paste into Claude (browser / chat)
3. Ask: *“Rewrite this into a precise, structured prompt for Claude Code”*
4. Add constraints (files, outputs, format)
5. Use the improved prompt in CLI

What improves automatically:

- Clear structure (steps, sections)
- Missing details filled in
- Edge cases + assumptions surfaced
- Output format specified

👉 *You describe intent → AI designs the instructions*

(you don’t need to be perfect, you just need to be clear about intent. AI handles the precision)

More tricks: using AI as pair programmer

Key patterns for iterating on code:

"Don't change existing functions."	→ Surgical changes - Claude only touches what you specify.
"Add a function that does X. Keep everything else."	→ Append-only - safe for scripts you're already relying on.
"Explain what line 47 does in plain English."	→ Code reading - understand before you trust.
"This is failing with: [error]. What's wrong?"	→ Debugging - paste the error, get a diagnosis.

You stay in control. You're directing what changes, what stays, what gets explained.
Claude doesn't get to freelance on your codebase.

Prompt 4 - Explain mode

From the analysis and the generated figures:

1. Write 3 bullet points suitable for a conference abstract. Each bullet should be one sentence, start with a number or percentage, and describe a concrete finding. Academic tone. No jargon beyond what a civil engineering audience would expect.

2. Recommend which of the four figures is most important to include in a journal submission and explain why in 2-3 sentences.

3. Identify the single most surprising finding from the terminal output - something that would prompt a reader to ask a follow-up research question. State the finding, then state the follow-up question it raises.

4. Write a one-paragraph interpretation of the geographic risk pattern shown in fig1_risk_map.png - where is risk concentrated, and what infrastructure or demographic factors might explain this?

Save to bridge_risk_demo/outputs/results_description.md

From results to presentation

Data

Your dataset, CSV, or simulated network

Analysis

Python script runs: stats, figures, tables

Script

generate_presentation.py reads results + figure files

PPTX

Ready-to-open conference deck with figures embedded

Start-to-PPTX in under 5-10 minutes. No slide-by-slide formatting.

What the presentation should include

Title slide	Institution branding, authors, conference name
Research Questions	Your specific Qs, formatted for presentation
Data & Methods	Dataset summary, methodology overview
Key Results	Figures embedded - load curve, vulnerability plot, map
Discussion	Interpretation of findings, implications
Conclusions	Summary bullets, future work, acknowledgements
References	Key citations formatted

The point isn't that it's perfect. The point is that you're not starting from a blank slide.

Prompt 5 - Conference presentation (pptx)

Read: bridge_risk_demo/outputs/data_description.md , bridge_risk_demo/outputs/results_description.md , Figures: fig1_risk_map.png, fig2_age_condition.png, fig3_risk_distribution.png, fig4_county_risk.png

Write a **Python script using python-pptx** that generates a **10-slide academic conference presentation**, saved to: bridge_risk_demo/outputs/presentation.pptx

Slides Structure

1.Title Slide Title, authors, conference footer
2.Problem 4 concise bullets: structural deficiency, Texas scale/age, budget limits, lack of framework Notes: explain ASCE context and why Texas
3.Research Question Main question (large text) , 3 supporting sub-questions , Notes: emphasise reproducibility
4.Data Bullets: source, size (~54k bridges), fields, key variables Small table: DECK_COND_058, ADT_029, YEAR_BUILT_027 Notes: highlight open data
5.Methods (Risk Score) Show formula combining condition, age, traffic (0.50 / 0.25 / 0.25 weights) Brief explanation of each component Risk tiers: Low, Moderate, High, Critical , Notes: explain weighting rationale
6.Results – Geographic Distribution Insert fig1_risk_map.png Caption + 2 key insights Notes: spatial patterns
7.Results – Age vs Condition Insert fig2_age_condition.png Caption + 2 insights Notes: key finding (age ≠ condition)
8.Results – Distribution & Counties Insert fig3_risk_distribution.png and fig4_county_risk.png Short captions Notes: highlight critical counts and top county
9.Discussion ≤5 bullets: importance of condition data, prioritisation, transparency, replicability, limitation Notes: application to asset management
10.Conclusions 3 concise conclusions (from results) 1 future work item 4 APA references Notes: closing and reproducibility

Key Requirements Include **speaker notes (80–120 words per slide)** Use **clear academic tone** Keep bullets concise (≤12 words where specified) Insert figures in correct slides, centered

Generate the .pptx file via the script

AI scaffolds it. You shape it.

The generated deck is a structured starting point - not a finished product.

AI provides	Your expertise adds
Correct slide structure	**The story that connects the slides**
Figures from your code	**The interpretation of what they mean**
Bullet-point results	**The nuance, caveats, and implications**
Consistent formatting	**The judgment call on what to emphasise**

Developing a full lecture

WARM-UP

Context & motivation

Title slide
Real-world failures
The prioritisation problem
Workshop roadmap

Why it matters before how it works — activates prior knowledge and sets stakes (Ausubel, 1968)

TECHNICAL

Concepts & methods

NBI dataset structure
Risk components + formulas
Weighted composite score
Risk tiers + Texas stats
Results + key findings
Model limitations

Worked examples bridge abstract formula to concrete outcome (Sweller, cognitive load theory)

HANDS-ON

Explore & discover

Open bridge_risk_explorer.html
Task 1: default weights
Task 2: condition-heavy
Task 3: no-traffic model
Compare + discuss

Active learning outperforms passive lecture - students retain 75% of what they do vs 5% of what they hear (Dale's Cone)

OUTPUT

Reusable artefacts

17-slide lecture deck
bridge_data.json (54k bridges)
Interactive HTML explorer
Speaker notes per slide

One prompt generates all three - the lecture IS the workflow demonstration

Tell me and I forget. Show me and I remember. Let me do it and I understand.

(teaching philosophy)

Prompt 5 - Lecture developer assistant

Create **three files** for a *Texas Bridge Risk Assessment workshop*:

1. generate_lecture_pptx.py Use **python-pptx** to create lecture_bridge_risk.pptx (17 slides, widescreen) Audience: Civil Engineering students
Use same colour palette as conference slides Include **speaker notes on every slide**
Structure: Slides 1–4 (Intro): title, real-world failures, prioritisation problem, workshop steps **Slides 5–16 (Technical):** NBI dataset overview Risk components (condition, age, traffic) with formulas + examples Composite risk score (weighted) Risk tiers + Texas stats Results summary + key findings Visuals: fig1_risk_map.png, fig2_age_condition.png, fig4_county_risk.png Model limitations Discussion questions **Slide 17 (Hands-on):** Instructions using bridge_risk_explorer.html 3 tasks: default weights, condition-heavy, no-traffic comparison
Save to: lecture_materials/lecture_bridge_risk.pptx

2. export_bridge_data.py Load data from bridges_texas.csv using existing functions: load_and_clean, compute_risk Export all bridges to bridge_data.json Each record:[LONGDD, LATDD, condition_risk, age_risk, traffic_risk] Round to 4 decimals Compact JSON (no spaces) Print summary: number of bridges + file size

3. generate_html.py Create a **self-contained interactive HTML app**: bridge_risk_explorer.html
Features: Dark theme layout with left control panel + map Load and embed bridge_data.json directly into HTML Sliders for weights (condition, age, traffic) Risk tier filters (Low → Critical) Live stats update "Update Map" button recalculates scores
Map: Plotly scatter_mapbox. Colour scale: green (low) → red (high) Tooltip: location + component risks Risk formula recomputed dynamically . if access block for openstreet are plot, make them small size . For the interactive map, use carto-positron as the Plotly mapbox style (not open-street-map). Set marker size to 6.

Final Step
Run:
python lecture_materials/export_bridge_data.py
python lecture_materials/generate_lecture_pptx.py
python lecture_materials/generate_html.py
Confirm all outputs were created.

The Lawyer Story

Summer 2025 · Nevada, USA

→ A lawyer submitted a brief citing 6 prior court cases.

→ Names. Dates. Case numbers. Perfectly formatted.

→ The judge asked for the original documents.

✗ **They didn't exist. Every citation was invented by ChatGPT.**

✗ No warning. No disclaimer. Delivered with total confidence.

The sentence: go to law schools and warn lawyers about AI.

Hallucinations haven't gone away. They've gotten more convincing.

Why AI hallucinates - and why it's unfixable (so far)

A language model predicts plausible text. Not verified facts.

Plausible ≠ True

When the model doesn't have exact data, it doesn't stop.
It completes the pattern.

When you ask it something, it doesn't look up the answer. It constructs it - token by token - based on what statistically makes the most sense to come next.

Think of it as a lossy compression system:
it compressed billions of words into maths,
and when the exact fact isn't in the maths, it fills the gap.

Three types of hallucination

Factual

Wrong date, number, person.
Ask multiple models: you'll get different answers.

E.g. When was X album certified gold? - each model gives a different date.

Reference

Invented citations with real-looking authors, journals, dates.
The deadliest for academic work.

E.g. 'Verify these quotes from Ortega y Gasset' - Claude finds half are fabricated.

Context

Misreads a document you gave it.
Adds or removes meaning that isn't there.

E.g. 'List all 10 principles of ISO 9241' - standard only has 7; model invents 3.

Privacy: know before you paste

Use AI freely with safe data, be cautious with anonymised data, and never share sensitive data without proper safeguards

Use case	Risk	Rule
Generating lectures, exams, exercises	Low	Use freely
Drafting feedback on anonymised student work	Medium	Anonymise first - no names or IDs
Real student data: names, IDs, grades	**HIGH**	**Never in a cloud AI tool without approval**
Sensitive research / embargoed results	**HIGH**	**Check your institution's data policy**

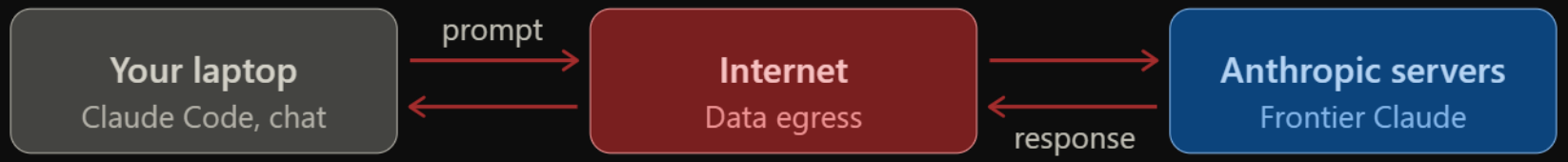
Local-model alternative: run open weights (Llama 3.3, Qwen 3.5, GLM-5) via Ollama — data stays on your machine. Claude Code uses Anthropic's API; for local-only workflows use Continue.dev, aider, or ***Ollama*** directly.

Privacy: know before you paste

Local-model alternative

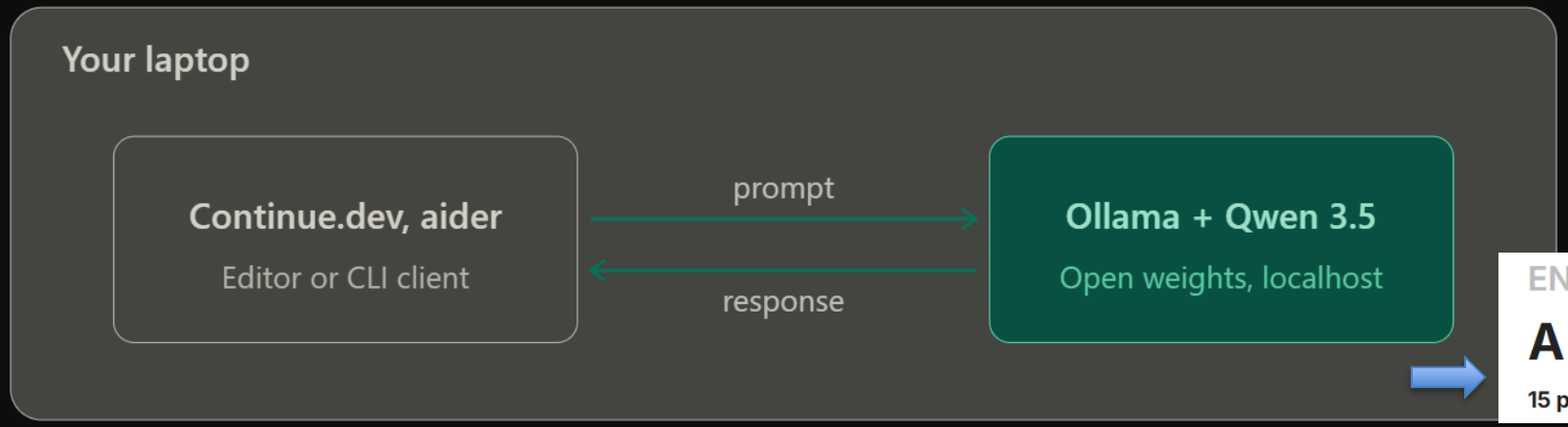
Cloud API

Prompt leaves your machine



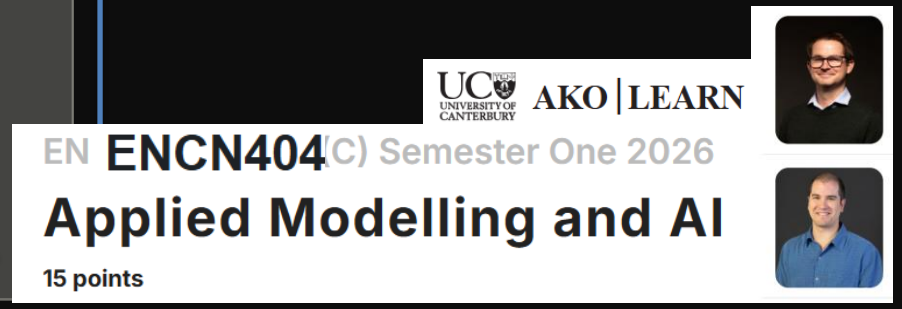
Local model

Prompt stays on your machine



Dept-level impact (internal use):

Governance → clear rules for internal AI use
Infrastructure → secure local + optional cloud setup
Cost control → track and manage usage
Reusability → shared academic & admin workflows
Compliance → data stays within the institution
Capability → staff trained for responsible internal use



What this means for academic/scientific use

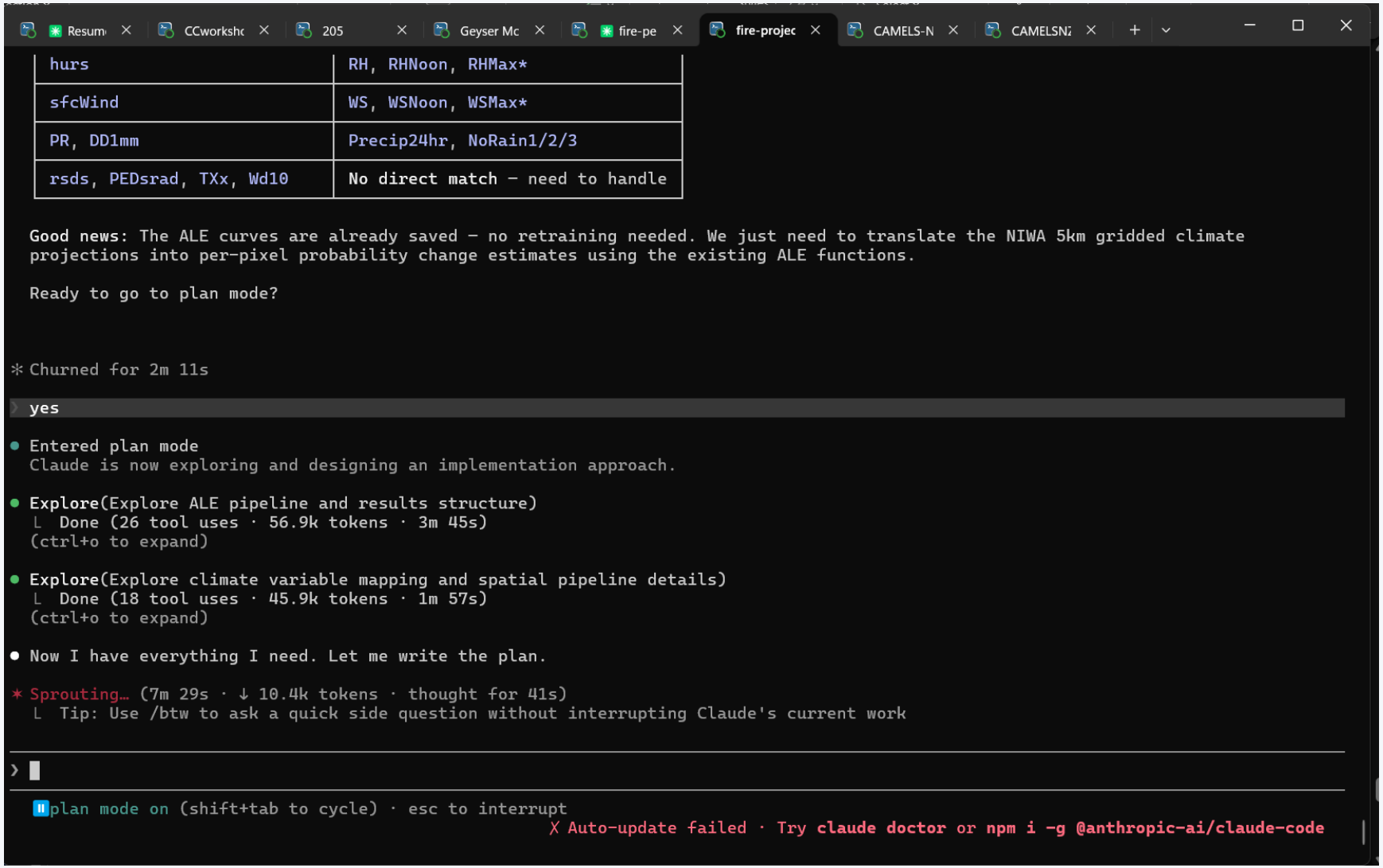
ALWAYS verify references	If Claude Code cites a paper, find it yourself. It may not exist.
Never trust code unreviewed	Run it. Read it. Understand what it does before using results.
Cross-check specific numbers	Dates, statistics, measurements - verify from primary sources.
Use for structure, not facts	Scaffolding, formatting, rewriting - excellent. Factual claims - verify.
Iterate with your expertise	The model hallucinates less when you give it the facts and ask it to structure them.

Prompt 6 - Check lecture and improve (if need it)

Read through the lecture materials in this folder and check them for anything that's unclear, out of date, inconsistent across slides, or missing context a student would need. Flag anything factual that looks wrong. Don't rewrite anything yet; just give me a prioritised list of what you'd change and why, so I can decide what's worth fixing.

Finally – Backup! We can save it, for instance, in GitHub.

How do I used it in my day to day?



Each tab is a **research assistant** (agent) working on a different project.

My role has become **supervising** each agent on a daily basis.

I'm not **focusing on** technical work (labor), but rather on **thinking-providing guidelines**

and deciding **what is relevant and what is not** (separate the wheat from the chaff).

The uncomfortable truth

You're not competing with the machines.

You're competing with the humans who use them.

Because the rules of good work (**clear thinking, strong judgment, domain expertise**) haven’t disappeared. The game hasn't changed completely. The speed has.

The comfortable truth

Professionals with sound judgment are best equipped to enhance their work with AI.

There's no better time to be an academic.

For discussion:

- What is being done to **solve the hallucination problem?**
- Which models lie the most?
- Understanding **Claude Code Cost**
- **Skills**: Reusable AI commands

Future workshops:

- Literature review with CC
- Scientific writing with CC
- Exam development and marking with CC

Claude Code: a professional CLI tool that puts AI inside your work environment, your files, your code, your research workflow.

Not a chatbot. A collaborator.

--

Dr Alberto Ardid
Lecturer / Assistant Professor
Department of Civil and Environmental Engineering
University of Canterbury, New Zealand

E319 |alberto.ardid@canterbury.ac.nz

How do I used it in my day to day?

New development workflows

Instead of:

write → compile → debug

We move toward:

describe → generate → validate → refine

Role evolves from:

“person who writes code”

to:

“person who ensures the right system gets built correctly”

Two worlds are forming

Group A

Hasn't figured out
voice-to-text on their phone.

Still arguing AI is
"just a fad."

Group B

Running small armies
of AI tools.

Doing in 1 hour what could
took a week.

↔ **Which side do you want to be on?**

Pareto rule: from 80/20 to 99/1

Making context stick: CLAUDE.md

text file your project remembers by.

- ✓ A CLAUDE.md file in your project root is auto-loaded every session
- ✓ Run */init* to have Claude scan the project and draft it for you
- ✓ Put architecture, conventions, key decisions, and gotchas ; keep it tight

Understanding Claude Code Costs

Three key cost drivers

- 🔢 **Token volume** — A typical session uses 10k–100k+ tokens depending on codebase size and iterations
- 🤖 **Model choice** — Sonnet ($3/M input) vs Opus ($5/M input) — a 67% cost difference per token
- 💾 **Cache hit rate** — Over 90% of tokens in real workflows are cache reads, charged at just 10% of input price

	Haiku	Sonnet	Opus
Input (per 1M)	$1	$3	$5
Output (per 1M)	$5	$15	$25

Here we used 50k-token: estimated ~$0.15

Claude Code need Claude membership: starting from 20USD (per month)

In-Confidence

Claude Code: Cost per Session

Three key cost drivers

🔢 **Token volume** — A typical session uses 10k–100k+ tokens depending on codebase size and iterations

🤖 **Model choice** — Sonnet ($3/M input) vs Opus ($5/M input) — a 67% cost difference per token

💾 **Cache hit rate** — Over 90% of tokens in real workflows are cache reads, charged at just 10% of input price

*Using **Claude Sonnet 4.6** (larger than Haiku, smaller than Opus). In this session we used **45k tokens**.*

Worst case — **~$0.14**
All fresh input tokens — *45k × $3/M*

Realistic — **~$0.03**
90% cache reads — *Blended effective rate*

Best case — **~$0.01**
Near-full cache hit — *Cache reads @ $0.30/M*

Sonnet 4.6 Token Pricing

Token Type	Rate (per 1M)
Fresh input	$3.00
Output	$15.00
Cache write	$3.75
Cache read	$0.30

Key insight

Cache reads dominate (~90% of tokens) - cutting effective cost by ~10×.

A typical 45k-token session costs just $0.1–$0.15 USD.

Sonnet: $3/$15 per 1M input/output • Cache reads: $0.30/M • Source: Anthropic API Docs

In-Confidence

What is being done to solve the hallucination problem?

Proposed solutions:

(1) Search agent: searches the web when it doesn't know something with certainty (perplexity)

(2) Think before you speak (slow and fast thinking): thinking model, reason with itself (chatGPT and Claude). Model generates an internal draft. However, thinking models tend to hallucinate more. More reasoning time does not equal truth.

(3) Context window: Context windows based on web searches (Gemini)

(4) Model orchestration (perplexity). Multiple agents that rephrase questions from different angles, and all search the web. A final agent aggregates everything.

Which models lie the most?

Based on benchmarks: (SimpleQE, Hallulens, etc.).

Most reliable: Perplexity
Most rigorous: Gemini (Source: Google)
Best with recent data: Grok (Source: Twitter)
Says it doesn't know when it doesn't: Claude
Most likely to lie: ChatGPT (reasoning models)
Least reliable: DeepSeek.

Skills - reusable AI commands

DEMO 4

A Skill is a named instruction set you define once and invoke forever.

/generate_lecture	topic + level + duration → full lecture with LOs, timing, questions
/generate_exam	course topic + marks → MCQ, short answer, extended response, marking guide
/literature_review	list of papers → structured review with themes and gaps
/dataset_analysis	data description → analysis plan + starter code

Think of Skills as macros - but for your entire academic workflow. Define once. Share across your team. Reuse forever.